

Reviewer Pack — Non-backtracking Spectral Gap Lower-Bounds Tseitin DPLL Tree...

Entry d1d1e2ec5d2a · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 09:37:32 UTC

Non-backtracking Spectral Gap Lower-Bounds Tseitin DPLL Trees

Entry ID: d1d1e2ec5d2a **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-08 09:37:32 UTC

1. Conjecture

Field A (mathematical branch): Ihara zeta / non-backtracking (Hashimoto) spectrum of regular graphs (Ihara 1966, Bass 1992, Friedman 2008, Alon-Boppana for $B(G)$) — an expansion proxy strictly sharper than the ordinary Laplacian gap and essentially absent from the proof-complexity literature **Field B** (complexity object): Resolution / DPLL refutation length $L_R(\text{Tseitin}(G, \sigma))$ of 3-regular Tseitin formulas with odd vertex charge $\sigma: V \rightarrow \mathbb{F}_2$ (the Urquhart / Ben-Sasson-Wigderson canonical sub-Frege gateway target)

Statement:

Let G be a connected 3-regular graph on n vertices and let $B(G) \in \{0,1\}^{\{2|E| \times 2|E|\}}$ be the non-backtracking (Hashimoto) matrix defined by $B[(u,v),(v,w)] = 1$ iff $w \neq u$, with second-largest absolute eigenvalue $\lambda_2(B)$. Define the poly-time graph invariant $\nu(G) := n \cdot \max(0, \sqrt{2} - (\lambda_2(B) - \sqrt{2}))/\sqrt{2}$, which is $O(1)$ for any G with $\lambda_2(B) \geq 2\sqrt{2}$ (the non-expander regime, including disconnected and tree-like G) and $\Theta(n)$ for Ramanujan-quality 3-regular expanders where $\lambda_2(B) \approx \sqrt{2}$. Conjecture: for every odd σ , $\log_2 T_{\text{DPLL}}(\text{Tseitin}(G, \sigma)) \geq (1/16) \cdot \nu(G)$, where T_{DPLL} is the leaf count of unit-propagation DPLL with fixed lexicographic branching; one trial with $\log_2 T < \nu/16 - 2$ falsifies.

Rationale (proposer's reasoning):

Friedman's theorem identifies $\sqrt{d-1} = \sqrt{2}$ as the true Alon-Boppana threshold for 3-regular non-backtracking spectra, and exactly this gap controls the mixing-time of cycle covers that Ben-Sasson-Wigderson width arguments require to lift from vertex-expansion to Resolution-length lower bounds. Standard $\lambda_2(L)$ -based invariants (Algebraic Connectivity, Sparsest-Cut LP) blur small-set expansion with bottleneck cuts, whereas $\lambda_2(B)$ is sensitive only to genuine Ramanujan-style mixing — which is precisely the regime Tseitin is hard. The Hashimoto spectrum is computable in $O(|E|^3)$ and has never been used as a proof-complexity invariant, yet it is the natural quantitative refinement of 'expander' that the open characterisation problem demands.

Taxonomy category: META_COMPLEXITY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 1ddccd36d31211cc

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across 150 trials (5 sizes $\times$ 30 graphs), conjecture is SUPPORTED iff $\text{support_fraction} = \#\{\log_2 T \geq \nu/16 \text{ OR CAPPED}\}/150 \geq 0.85$ AND Spearman $\rho(\log_2 T, \nu)$ over uncapped trials ≥ 0.6 . FALSIFIED if any uncapped trial yields $\log_2 T < \nu/16 - 2$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - non-backtracking spectrum Tseitin formula resolution lower bound - Ihara zeta function expander DPLL proof complexity - Hashimoto matrix Ramanujan graph Tseitin Urquhart resolution width

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.0s

5.1 Generated Python source

```
import random
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def is_3_regular(G):
        degree = [0] * len(G)
        for u in range(len(G)):
            degree[u] = sum(1 for v in G[u] if v > u)
        return all(d == 3 for d in degree)

    def configuration_model(n, m):
        G = [[] for _ in range(n)]
        edges = set()
        while len(edges) < m:
            u = random.randint(0, n-1)
            v = random.randint(0, n-1)
            if u != v and (u, v) not in edges and (v, u) not in edges:
                G[u].append(v)
                G[v].append(u)
```

```

        edges.add((u, v))
    return G

def hashimoto_matrix(G):
    E = [(u, v) for u in range(len(G)) for v in G[u] if v > u]
    n = len(E)
    B = [[0] * (2*n) for _ in range(2*n)]
    for i, (u, v) in enumerate(E):
        j = E.index((v, u))
        B[2*i][2*j+1] = 1
        B[2*i+1][2*j] = 1
    return B

def eigenvalues(matrix):
    n = len(matrix)
    I = [[Fraction(1 if i == j else 0) for j in range(n)] for i in range(n)]
    A = matrix
    Q, R = [], []
    for _ in range(n):
        Q.append(A[0])
        r = [A[i][0] / A[0][0] for i in range(1, n)]
        R.append(r)
        A = [[A[i][j] - sum(Q[k][j] * R[k][i] for k in range(i)) for j in range(n)] for i in range(1, n)]
    Q = [q / q[0] for q in Q]
    R = [r / r[0] for r in R]
    eigenvals = []
    while len(eigenvals) < n:
        max_eigval = -float('inf')
        for i in range(n):
            if all(abs(Q[i][j]) < 1e-9 for j in range(i+1, n)):
                eigval = Q[i][i]
                eigenvals.append(eigval)
                Q = [[Q[j][k] - eigval * Q[j][i] * Q[i][k] / Q[i][i] for k in range(n)] for j in range(n)]
                break
        else:
            raise ValueError("Matrix is not diagonalizable")
    return eigenvals

def lexicographic_dp11(G, sigma):
    n = len(G)
    stack = [(0, 0)]
    solved = False
    cap = 2**22
    while not solved and len(stack) < cap:
        u, i = stack[-1]
        if i == len(G[u]):
            stack.pop()
        else:
            v = G[u][i]
            if sigma[v] == 0:
                stack.append((v, 0))
            elif sigma[v] == 1:
                solved = True
    return solved

def tseitin_encoding(G, sigma):

```

```

n = len(G)
clauses = []
for u in range(n):
    for v in G[u]:
        if sigma[v] == 0:
            clauses.append([-(u+1), -(v+1)])
        elif sigma[v] == 1:
            clauses.append([(u+1), (v+1)])
return clauses

def count_leaves(clauses, cap):
    n = len(clauses)
    stack = [(0, 0)]
    solved = False
    while not solved and len(stack) < cap:
        u, i = stack[-1]
        if i == len(clauses[u]):
            stack.pop()
        else:
            clause = clauses[u][i]
            if all(abs(x) in [1, -1] for x in clause):
                solved = True
            else:
                stack.append((u, i+1))
    return solved

def generate_graph(n):
    while True:
        G = configuration_model(n, n*3)
        if is_3_regular(G):
            return G

def compute_nu(G):
    B = hashimoto_matrix(G)
    eigenvals = sorted(eigenvalues(B), reverse=True)
    lambda2 = eigenvals[1]
    nu = n * max(0, (2**0.5 - (lambda2 - 2**0.5)) / 2**0.5) / 2**0.5
    return nu

def run_dpll(G, sigma):
    clauses = tseitin_encoding(G, sigma)
    cap = 2**22
    solved = count_leaves(clauses, cap)
    if not solved:
        return False, cap
    else:
        return True, None

n_values = [12, 16, 20, 24, 28]
trials = 30
support_threshold = 0.85
rho_threshold = 0.6
total_trials = len(n_values) * trials
nu_respecting_count = 0
log2T_values = []

```

```

for n in n_values:
    for _ in range(trials):
        G = generate_graph(n)
        sigma = [random.choice([0, 1]) for _ in range(n)]
        solved, cap = run_dpll(G, sigma)
        nu = compute_nu(G)
        log2T = Fraction(cap).log(2) if cap is not None else 0
        log2T_values.append((n, log2T, nu))
        if solved or (cap is not None and log2T >= nu / 16):
            nu_respecting_count += 1

support_fraction = nu_respecting_count / total_trials
rho = 0.0

uncapped_trials = [(n, log2T, nu) for n, log2T, nu in log2T_values if cap is not None]
if len(uncapped_trials) > 0:
    from scipy.stats import spearmanr
    _, rho = spearmanr([log2T for n, log2T, nu in uncapped_trials], [nu for n, log2T, nu in uncapped_trials])

result = {
    "metric_name": "log2T/v(G)",
    "metric_value": sum(log2T / nu for n, log2T, nu in log2T_values) / len(log2T_values),
    "instances_tested": total_trials,
    "conjecture_holds": support_fraction >= support_threshold and rho >= rho_threshold,
    "counterexample": "" if support_fraction >= support_threshold and rho >= rho_threshold else "
}

return result

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.randint(1, 1000000) for _ in range(30)]
    for seed in seeds:
        print(f"TRIAL: {run_trial(seed)}")

log2T_values = [log2T for n, log2T, nu in log2T_values]
support_fraction = sum(nu_respecting_count / total_trials)
rho = 0.0

uncapped_trials = [(n, log2T, nu) for n, log2T, nu in log2T_values if cap is not None]
if len(uncapped_trials) > 0:
    from scipy.stats import spearmanr
    _, rho = spearmanr([log2T for n, log2T, nu in uncapped_trials], [nu for n, log2T, nu in uncapped_trials])

result = {
    "metric_name": "log2T/v(G)",
    "metric_value": sum(log2T / nu for n, log2T, nu in log2T_values) / len(log2T_values),
    "instances_tested": total_trials,
    "conjecture_holds": support_fraction >= support_threshold and rho >= rho_threshold,
    "counterexample": "" if support_fraction >= support_threshold and rho >= rho_threshold else "
}

print(f"RESULT: {'SUPPORTED' if result['conjecture_holds'] else 'FALSIFIED'} mean={result['metric_value']}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out after 240s (returncode 124) before producing any RESULT line, so neither the support nor falsification condition can be evaluated. | next: Reduce the largest size tier and/or cap DPLL leaf counts more aggressively, then rerun with a longer timeout to obtain the 150 trials required by the pre-registered criterion.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	90286
2	preregistration	claude_max	opus	0	0	5381
3	novelty	claude_max	opus	0	0	2900
4	novelty	claude_max	opus	0	0	6488
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17382
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	22029
7	judge	claude_max	opus	0	0	4784

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 149250 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/d1d1e2ec5d2a.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/d1d1e2ec5d2a.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/d1d1e2ec5d2a.tar.gz` (if generated)